# 김종언

**KIM JONGEON**

010.4270.9729
vocolate17@gmail.com

## 소개

- 컴퓨터학부 전공으로 CS 기초를 익히고, 클라우드운영 마이크로전공으로 운영 관점을 넓혔습니다.
- AWS Cloud School 1,050시간 교육으로 AWS · Docker · Kubernetes · Terraform · CI/CD 역량을 체계화했습니다.
- 자동확장 · GitOps · AI 서비스 인프라 프로젝트에서 배포 · 관측 · 장애 분석 역량을 검증했습니다.

## 학력 · 교육

| | |
|---|---|
| 2019.03 - 2026.08 | 한양대학교 ERICA 컴퓨터학부 (학사)<br>클라우드운영 마이크로전공 이수 |
| 2025.12 - 2026.07 | AWS Cloud School 13기 \| 1,050시간<br>Python · Linux · 네트워크 · AWS · Docker · Kubernetes · CI/CD · Terraform |
| 2025.07 - 2025.08 | 코멘토 IT 직무부트캠프 \| 5주<br>AWS VPC · EKS · EFK 로그 수집 · CloudWatch 알림 · 운영 문서화 실습 |

## 주요 프로젝트

2026.05 - 2026.07

### BADA | 임금체불 증거 패키징 서비스

**역할: Infra · CI/CD · IaC**

- 계약서 · 급여명세서 · 녹취 등 흩어진 자료를 AI로 구조화해 Evidence Pack PDF로 만드는 서비스
- Terraform과 GitHub Actions OIDC로 인프라 변경 · 배포 검증 흐름 구성
- ECS Fargate · SQS/DLQ로 장기 AI 작업을 분리하고, 1,000 VU 실험에서 CPU peak 99.75%→60.35% 확인

2026.03 - 2026.04

### Multi-Region EKS GitOps

**역할: Osaka Staging First Mover · GitOps 환경 검증**

- 서울 Production · 오사카 Staging · 도쿄 DR의 3리전 EKS GitOps 환경 구성
- PVC · ECR 권한 · Private RDS 이슈를 해결하고, Kustomize로 환경 차이 분리
- Pod 강제 삭제 시 RTO 약 6초 · DB 영향 없는 RPO 0 확인, 이슈 11건 문서화

2026.02 - 2026.03

### Docker Swarm Auto Scaling

**역할: CI/CD · Shell/Prometheus 자동화 · 실험 설계 · 수행**

- EC2 3대 Docker Swarm에서 GitHub Actions→ECR→Swarm 배포 흐름 구성
- Prometheus API→Shell 및 CloudWatch Alarm→SNS scale-out 경로 구현
- 4개 조건×10회, 총 40회 반복 측정으로 감지 구조별 반응 · 관리성 trade-off 검증

## 기술 역량 · 자격증

| | |
|---|---|
| **System · Network** | **Linux · Bash · TCP/IP · Subnet · Routing · DNS** |
| **Cloud · Container** | **AWS · Docker · Kubernetes** |
| **IaC · CI/CD** | **Terraform · GitHub Actions · OIDC · Argo CD · Kustomize** |

**AWS Certified Solutions Architect - Associate (SAA-C03) | 2026.06**

**TOEIC Speaking Test | Intermediate High (150) | 2026.08**

KIM JONGEON · PORTFOLIO

# 포트폴리오


## 김종언
KIM JONGEON

CONTACT
vocolate17@gmail.com
010.4270.9729

# 목차


01 **소개** — Page 3
학습 흐름과 기반 역량

02 **이력** — Page 4
전공 · 교육 · 활동 이력

03 **프로젝트 목록** — Page 5
주요 프로젝트 한눈에 보기

04 **BADA** — Page 6–23
AI 증거 패키징 서비스 · 인프라 · 검증

05 **Multi-Region EKS GitOps** — Page 24–31
3리전 EKS · GitOps 배포 · DR 검증

06 **Docker Swarm Auto Scaling** — Page 32–38
Shell · CloudWatch 반응속도 정량 비교

07 **마무리** — Page 39

PROFILE

# 소개

## 김종언

KIM JONGEON

이메일 vocolate17@gmail.com

연락처 010.4270.9729

**한양대학교 ERICA 컴퓨터학부**

학사 졸업 · 2026.08


## 기초를 쌓고,
## 운영의 흐름을 검증했습니다.

01
### 기초 학습
Linux · Network · Server

02
### 클라우드 기반
AWS 구축·운영 프로젝트

03
### 컨테이너 환경
Docker · Kubernetes 적용

04
### 자동화 경험
Terraform
CI/CD · GitOps

HISTORY

# 이력

2026

2025

2024

2019

## 학력 / 교육이력

**2026.08**
### 한양대학교 ERICA 컴퓨터학부 · 졸업

**2025.12 – 2026.07**
### AWS Cloud School 13기 · 1,050시간
Linux · AWS · Docker · Terraform · CI/CD
AWS 공식 교육 8개 과정 이수

**2025.07.25 – 08.23**
### 코멘토 IT 직무부트캠프
AWS 구축·운영 실무

**학부 과정 병행**
### 클라우드운영 마이크로전공

**2019.03**
### 한양대학교 ERICA 컴퓨터학부 · 입학

## 활동 / 주요 프로젝트

**2026.05 – 07**
### BADA
AWS Cloud School 13기 최종 프로젝트

**2026.03 – 04**
### Multi-Region EKS GitOps
3개 리전 EKS · GitOps 운영 · DR 구조

**2026.02 – 03**
### Docker Swarm Auto Scaling
Shell Script · CloudWatch 반응속도 비교

**2024.08 – 2025.01**
### PickSell 창업동아리
교내 창업동아리 활동

**2024.08 – 2025.06**
### Chewing Class
고트헤이븐 산학캡스톤 프로젝트

PROJECTS

# 프로젝트 목록

01

## BADA

임금체불 증거 패키징 서비스

기간

2026.05 – 07

02

## Multi-Region EKS GitOps

3개 리전 EKS · GitOps 운영 · DR 구조

기간

2026.03 – 04

03

## Docker Swarm 기반 Auto Scaling 반응속도 비교 분석

Shell Script와 AWS CloudWatch 방식의 반응속도 정량 비교 분석

기간

2026.02 – 03

# 내 기록을 모아,
# 받을 준비.

취약 근로자를 위한 임금체불 증거 정리 앱

**BADA**

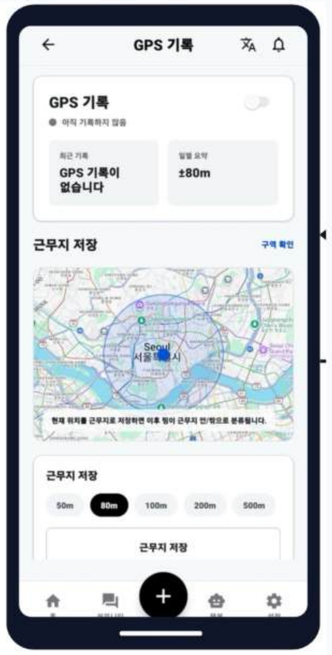
# 개요

취약근로자의 임금·근로 증거를 수집 및 분석하고 PDF 증거 패키지로 제공하는 AI 기반 모바일 서비스

## 다국어 번역

## OCR / 음성전사

## GPS



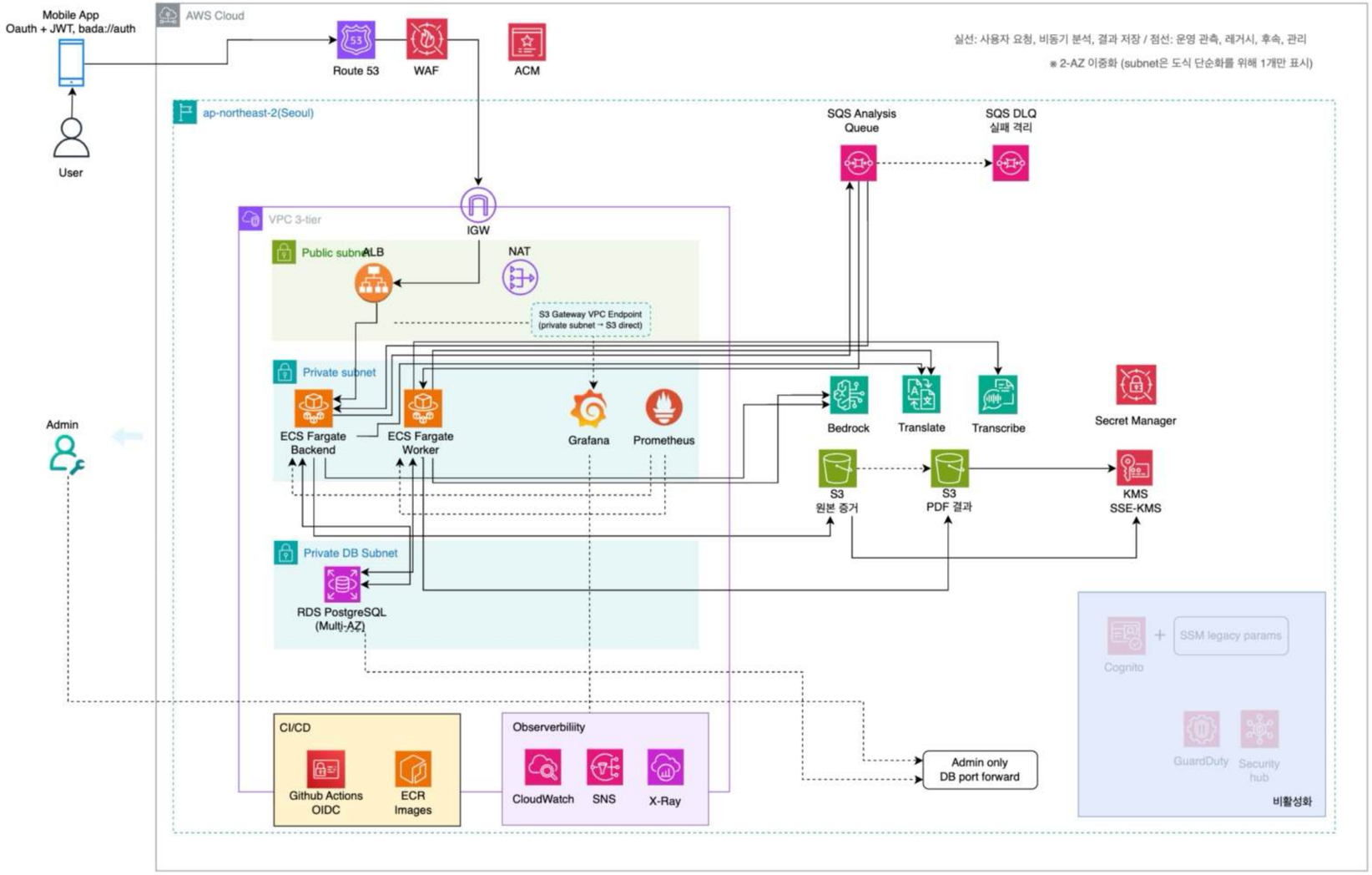

## AI 챗봇

## PDF 증거 패키지

**다국어 안내부터 증거 수집·분석, 상담 준비까지 하나의 사건 흐름으로 연결합니다.**

PROJECT 01 | BADA · INFRA & IaC

# 기여한 부분 (1): Terraform 기반 AWS 인프라 설계



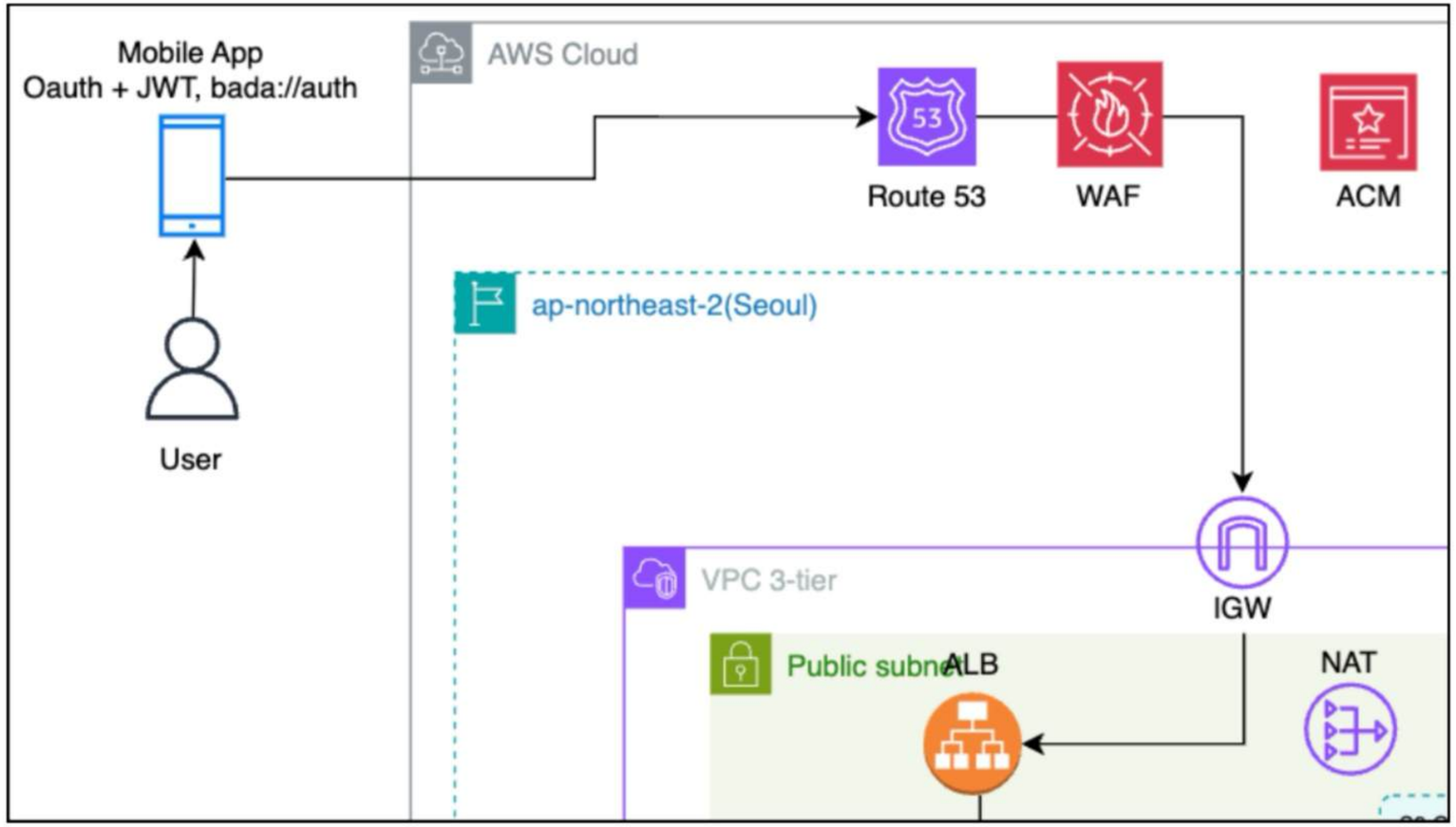

1 **진입 · 보안**
Route 53 · ACM · WAF
ALB (HTTPS 443)

2 **서비스 계층**
ECS Fargate 4개
private subnet · 2 AZ
Auto Scaling

3 **데이터 · AI**
SQS·DLQ 비동기 · Bedrock
RDS Multi-AZ · S3+KMS

4 **운영 · 자동화**
GitHub Actions · Terraform
Plan-in-PR · 관측 · 보안

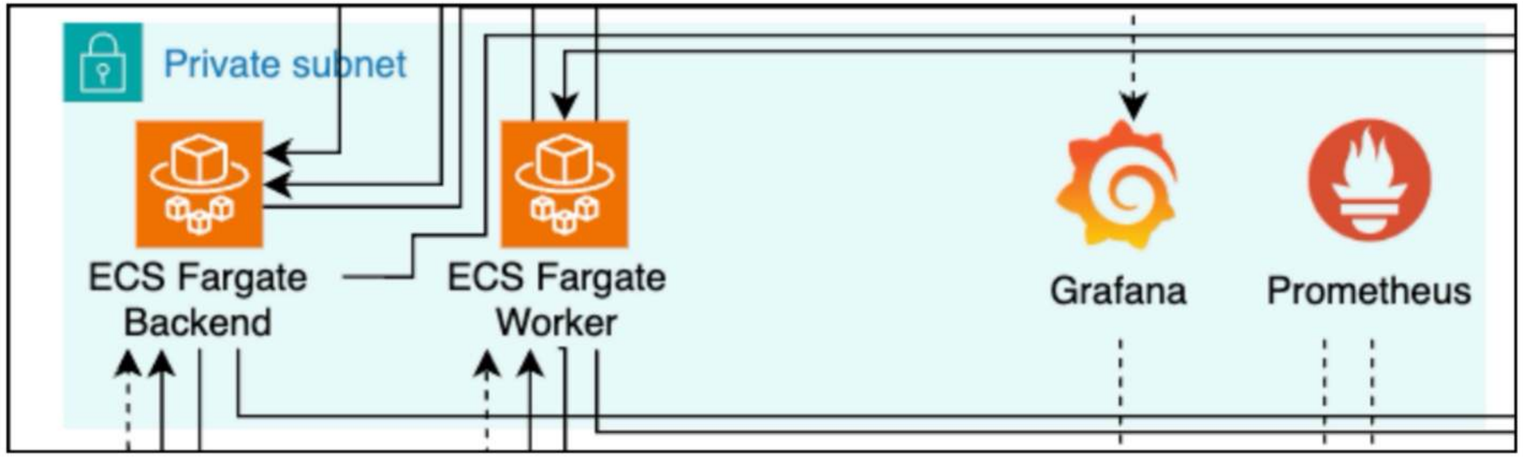
# 진입 계층




## ① 진입

- HTTPS 단일 진입점 구성
- WAF + ALB로 외부 요청 보호
- Host Routing 활용 :
  API / Grafana 분리
- Dev · Prod 도메인과 state 분리
- 모바일 딥링크 인증 흐름 지원

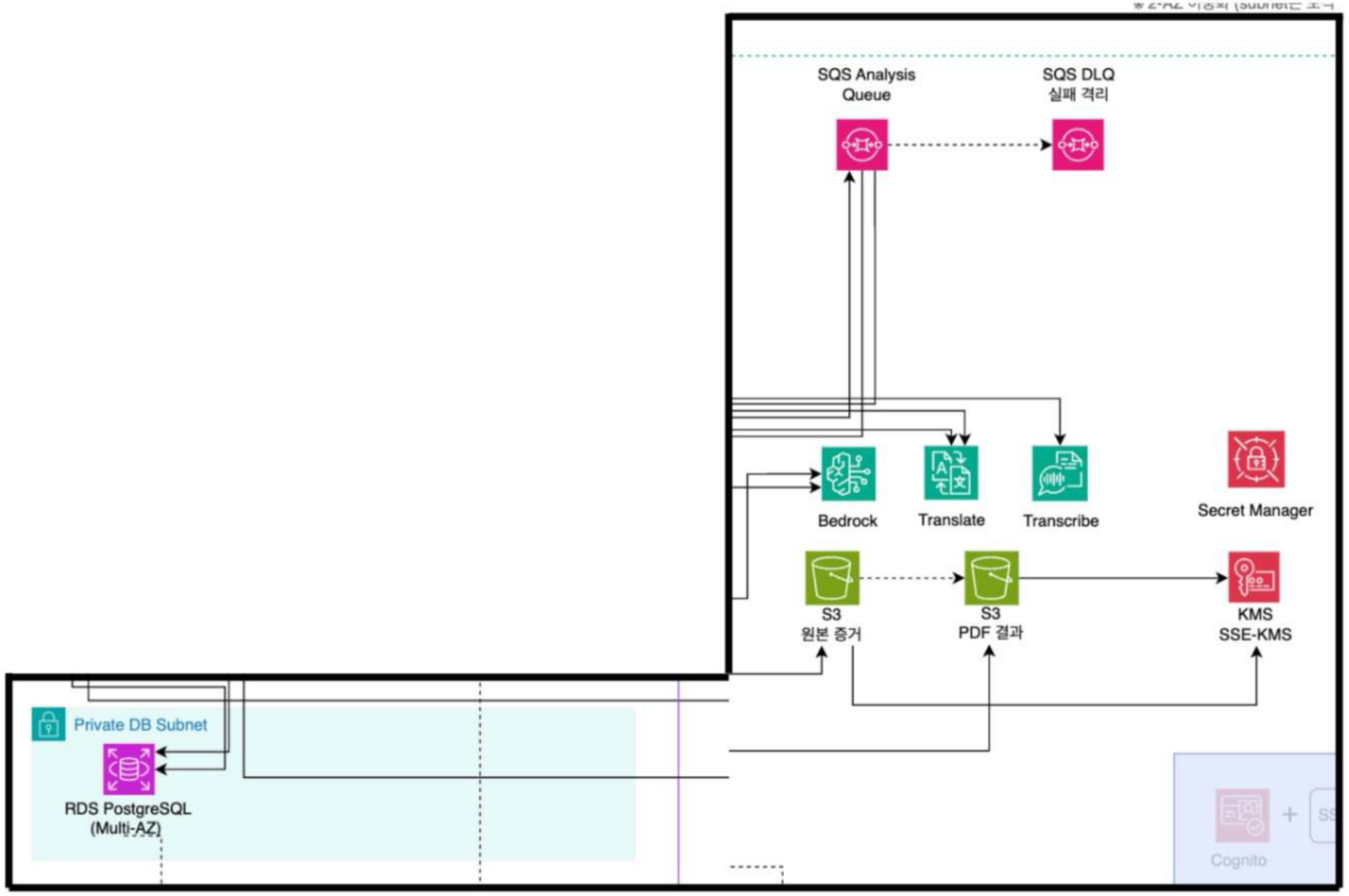
# 서비스 계층




## ② 서비스

- ECS Fargate 4개
- Backend / Worker / Monitoring 역할 분리
- Backend : API / 작업 등록
- Worker : 분석 / STT / 증거 PDF 생성 비동기 처리
- 2개 AZ 분산으로 가용성 확보

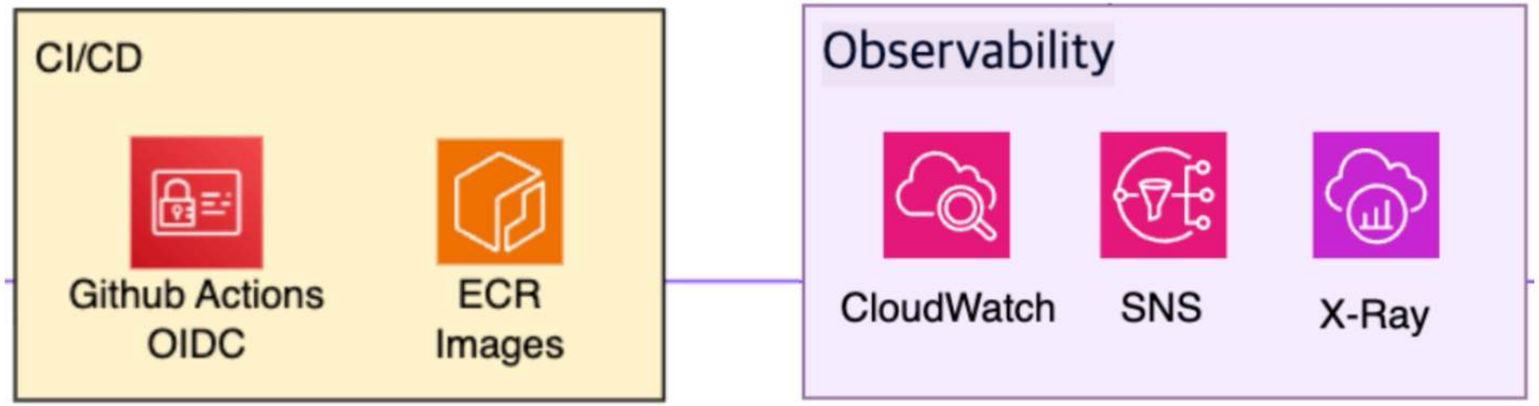
# 데이터 · AI




## ③ DB & AI

- RDS PostgreSQL + PostGIS를 Private Subnet에 격리
- SQS + DLQ로 분석 작업 재시도·실패 격리
- Bedrock / Transcribe / Translate
- Evidence / Report S3 버킷 분리
- SSE-KMS / Secrets Manager로 데이터 보호

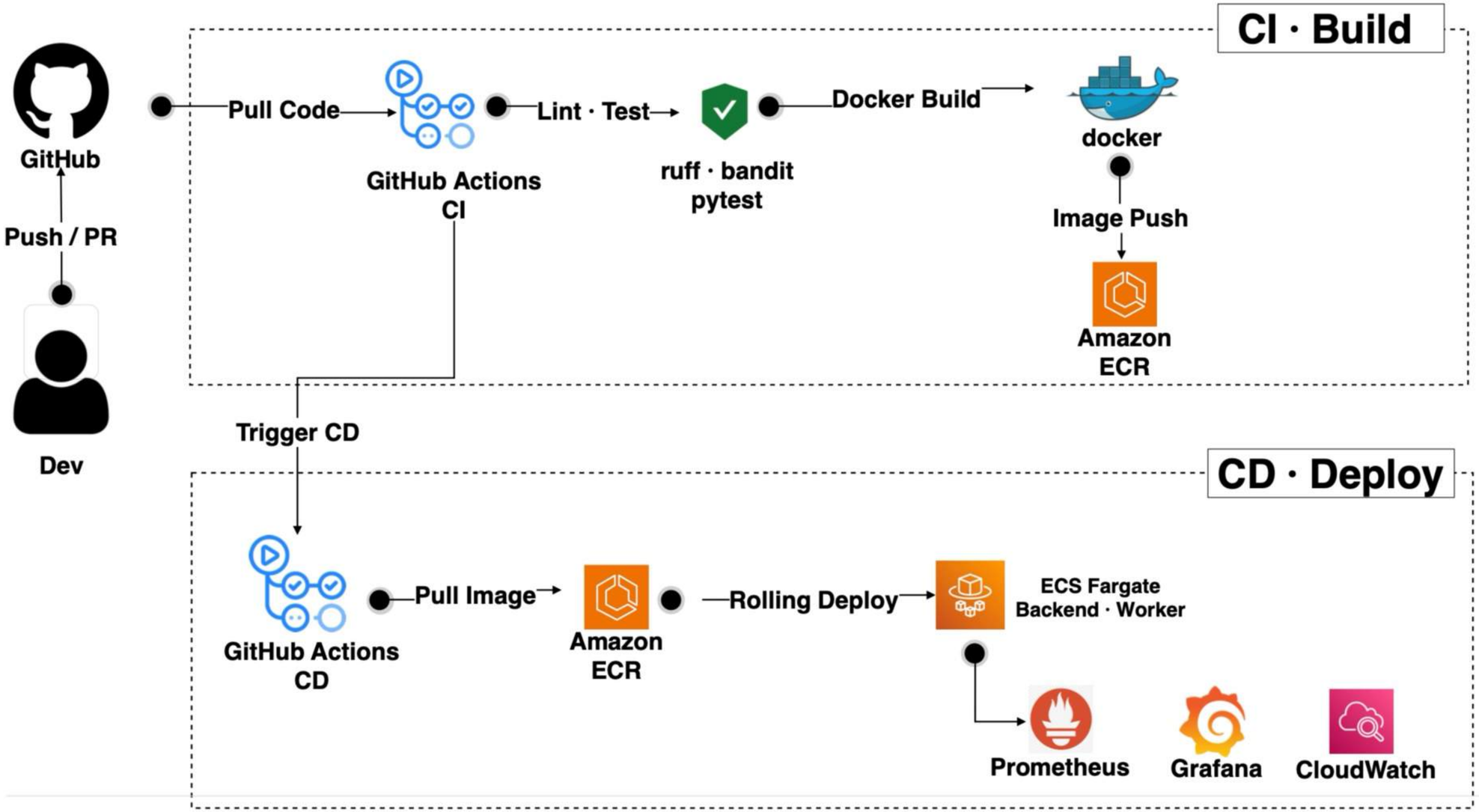
# 운영 관리 계층




## ④ 운영 · 관리

- GitHub Actions OIDC 기반 배포
- ECR → ECS Task Definition 자동 갱신
- HTTPS Health Check 통과 후 배포
- CloudWatch + SNS로 장애 알림
- Grafana로 서비스 상태 상시 관측
- 롤백 절차 문서화

# 기여한 부분 (2): CI/CD 설계 — 자동 배포, 승인 기반 통제



PROJECT 01 | BADA · CI/CD

# CI/CD 상세 (1/2)

코드 변경부터 운영 검증까지 자동화

1
**Push / PR**
GitHub Actions
OIDC 단기 권한

›

2
**품질 게이트**
pytest · ruff · bandit
코드·보안 검증

›

3
**이미지 빌드**
Docker Build
ECR Push

›

4
**ECS 배포**
Task Revision
Rolling Deploy

›

5
**배포 게이트**
ALB /health 200
ECS Stable

✓ **보안**
장기 Access Key 미사용

✓ **안정성**
배포 실패 시 자동 롤백

✓ **검증**
운영 endpoint 상태 확인

# CI/CD 상세 (2/2)

자동화 범위와 승인 기준을 분리

## Mobile Build

1. **수동 실행** — workflow_dispatch
2. **EAS Build** — Android APK / AAB
3. **테스트 배포** — 필요 시 실행

**불필요한 빌드를 줄이고 테스트 배포 시점 통제**

## Infra Control

1. **PR Plan** — fmt · validate plan
2. **담당자 승인** — Apply 수동 실행
3. **환경 분리** — dev/prod state tfvars

**인프라 변경은 자동 검증, 실제 반영은 승인 기반**

✓ **자동화**
Backend·Worker 반복 배포

✓ **보안**
OIDC 기반 최소 권한

✓ **통제**
Apply·앱 빌드 승인

PROJECT 01 | BADA · TROUBLESHOOTING

# 트러블 슈팅: Terraform 의존성 레이스 해결

Terraform 리소스 생성 순서를 명시해 ECS 배포의 초기화 위험을 줄였습니다.

infra/compute.tf

```
@@ -375,7 +375,12 @@ resource "aws_ecs_service" "backend" {
375  375
376  376     health_check_grace_period_seconds = 60
377  377
378      -   depends_on = [aws_lb_listener.http]
     378 +   depends_on = [
     379 +     aws_lb_listener.http,
     380 +     aws_lb_listener.https,
     381 +     aws_lb_listener_rule.api,
     382 +     aws_secretsmanager_secret_version.app
     383 +   ]
```

기존 HTTP Listener 단일 의존성 → ALB 라우팅·Secret Version까지 선행 조건 확장

## 문제 상황

Terraform이 독립 리소스를 병렬 생성하는 과정에서
ECS 서비스가 ALB 라우팅과 Secret Version 준비 전에
생성될 수 있는 의존성 레이스를 확인했습니다.

## 해결 및 검증

depends_on으로
ALB Listener / API Rule / Secret Version을
ECS 서비스의 선행 조건으로 명시했습니다.
PR #245에서 Terraform Plan과 품질 및 보안 검증을
통과해, 변경 전 검토 가능한 IaC 배포 절차로 관리했습니다.

## 배운 점

IaC는 리소스를 만드는 도구를 넘어,
서비스가 안전하게 기동할 수 있도록
의존성과 생성 순서까지 설계해야 한다는 것을 배웠습니다.

# 고도화 (1/3) - RDS Multi-AZ 안전 전환

기존 운영 DB를 직접 변경하지 않고, 별도 암호화 Multi-AZ DB를 검증한 뒤 연결만 전환했습니다.

## 1. 분리된 암호화 DB 생성

infra/data.tf

```
+ resource "aws_db_instance" "postgres_rehearsal" {
+   count = var.enable_rehearsal_multiaz_db ? 1 : 0
+
+   identifier              = var.rehearsal_db_identifier
+   engine                  = "postgres"
+   engine_version          = "16"
+   instance_class          = var.db_instance_class
+   allocated_storage       = var.db_allocated_storage
+   storage_encrypted       = true
+   kms_key_id              = var.rehearsal_db_kms_key_arn
+   db_name                 = "bada"
```

기존 운영 DB와 분리된 PostgreSQL 16 기반 RDS 생성

## 2. Private Multi-AZ 구성

```
+   publicly_accessible = false
+   multi_az            = true
+
+   tags = merge(local.common_tags, {
+     Name    = var.rehearsal_db_identifier
+     Purpose = "multi-az-rehearsal"
+   })
+ }
```

외부 접근 차단 · Multi-AZ·암호화 적용

## 3. Cutover · Rollback 토글

```
+ locals {
+   app_db_address = (
+     var.use_rehearsal_multiaz_db_as_app_db
+     ? aws_db_instance.postgres_rehearsal[0].address
+     : aws_db_instance.postgres.address
+   )
+ }
```

앱 연결 전환 · 기존 DB 복귀 경로 확보

### 문제 상황

기존 Single-AZ·비암호화 DB는 장애 대응과
민감 증거 데이터 보호 측면에서 개선이 필요했습니다.

### 전환 전략

새 encrypted Multi-AZ DB를 별도로 생성하고
데이터 정합성·PostGIS·앱 연결을 검증한 뒤
전환했습니다.

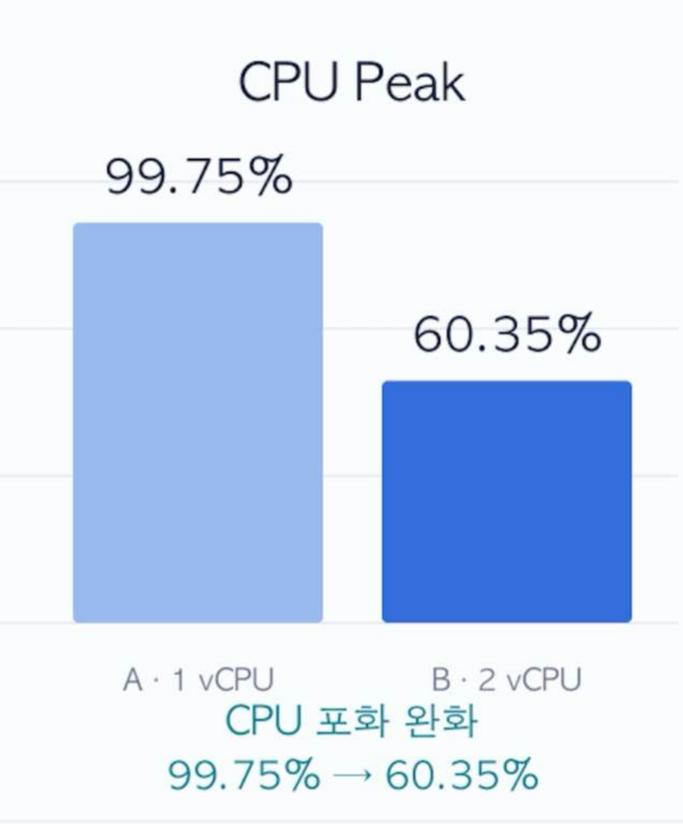
### 안전한 Cutover

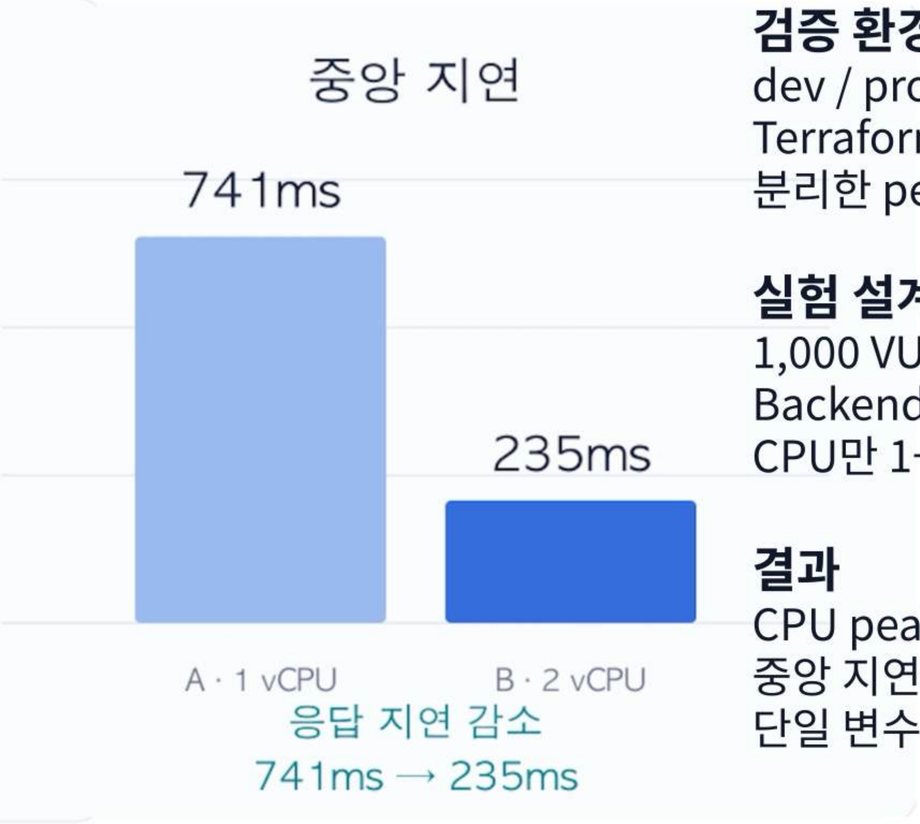
Secrets Manager의 DATABASE_URL 참조만
변경하고 기존 DB는 Rollback 경로로 보존했습니다.

PROJECT 01 | BADA · PERFORMANCE

# 고도화 (2/3) - 분리된 perf 환경과 병목 개선

분리된 perf 환경에서 단일 변수 A/B 실험으로 Backend CPU 병목 완화 효과를 검증했습니다.




**검증 환경**
dev / prod와
Terraform state·VPC·ECS·RDS·SQS를
분리한 perf 환경에서 실험했습니다.

**실험 설계**
1,000 VU 기준선에서 Backend CPU 포화를 확인했습니다.
Backend 4 tasks, 4GB/task·동일 RDS 조건을 유지하고
CPU만 1→2 vCPU로 변경했습니다.

**결과**
CPU peak는 99.75%→60.35%,
중앙 지연은 741ms→235ms로 개선되었습니다.
단일 변수 조정으로 병목 완화 효과를 검증했습니다.

PROJECT 01 | BADA · PERFORMANCE

# 고도화 (3/3) - 10,000 VU 한계 검증

10,000 VU 대규모 조건에서 성공률뿐 아니라 tail latency와 실패 원인을 함께 측정했습니다.

## 10,000 VU 한계 실험 - 결과 요약

2XX 성공

88.39% 1,036,422건

실패 합계

11.61% 5XX + timeout

Tail latency

p95 19.4초 runner 평균

### 검증 조건

20개 ECS Fargate k6 runner가 각 500 VU를 생성했습니다.
IP rate limit 영향을 배제하기 위해 /version 엔드포인트를 대상으로 실행했습니다.

### 처리 결과

총 1,172,609건을 요청했고, 평균 2,351.5 RPS를 기록했습니다.
2XX 응답은 88.39%였습니다.

### 실패 분석

HTTP 5XX 4.85%와 timeout·무응답 6.76%로 전체 실패율은 11.61%였습니다.

### 해석과 다음 조치

runner 기준 p95는 19.4초였습니다.
10,000 VU를 안정 처리했다고 볼 수는 없으며, 동시성·timeout·요청량 기반 autoscaling을 다음 최적화 대상으로 도출했습니다.

# 기술 스택

## Mobile / Frontend

Expo React Native · Expo Router · EAS Build
Next.js · TypeScript · next-intl

## Backend / AI

Python · FastAPI · SQLAlchemy · Alembic
Amazon Bedrock · Transcribe · Translate

## Data / Async

RDS PostgreSQL 16 · PostGIS · S3
SQS · DLQ · Secrets Manager · KMS

## Cloud / Infrastructure

ECS Fargate · ECR · ALB · VPC
Route 53 · ACM · WAF · Terraform

## CI/CD / Observability

GitHub Actions · OIDC · Docker
Prometheus · Grafana · CloudWatch · SNS · X-Ray

## Quality / Security

pytest · ruff · Bandit · pip-audit
IAM 최소 권한 · SSE-KMS · Terraform Plan

# Appendix

# AWS FinOps Agent - 비용 절감 및 모니터링

BADA의 비용 증가 원인을 계층별로 분석하고, 서비스 영향이 있는 변경은 검증과 승인 후 적용하도록 운영 기준을 만들었습니다.

01

## 비용 증가 원인 파악의 어려움

RDS / ECS / NAT / CloudWatch
Bedrock 비용이 함께 증가해,
어느 계층을 먼저 관리해야 하는지
빠르게 판단하기 어려웠습니다.

→

02

## 비용·이상·최적화 후보 분석

FinOps 분석 결과를 Cost Explorer,
Cost Anomaly Detection,
Optimization Hub API로
교차검증했습니다.

→

03

## 최적화 우선순위와 변경 통제

RDS 중지·RI·Graviton·gp3 전환 등
월 약 $43.01 절감 후보를 식별했습니다.
단, 서비스 영향이 있는 RDS 중지와
장기 약정 구매는 자동 적용하지 않고,
호환성·롤백 가능성 검토 후 승인하도록
통제했습니다.

비용 원인 불명확

분석 결과 검증

절감 후보 식별 → 영향도 검토 → 승인 후 변경

**월 약 $43.01**
**검증된 비용 절감 후보**

**RDS · ECS · NAT**
**우선 관리 대상 식별**

**자동 변경 0건**
**영향도 검토 후 승인 적용**

APPENDIX | BADA · DevOps Agent

# AWS DevOps Agent - 안정성 및 성능 복원

BADA의 과거 prod 배포 실패를 분석하고, CloudTrail과 운영 기록으로 검증해 재발 방지 Terraform 개선으로 연결했습니다.

01

## 배포 실패 원인 추적의 분산

prod 초기 배포 실패 시 ECS 이벤트, Secrets Manager, CloudTrail을 각각 확인해야 해 원인 탐색 범위가 넓었습니다.

분산된 운영 증거

**분석 전용 Agent Space: Tokyo**

→

02

## 관계 매핑과 장애 이력 분석

분석 전용 Tokyo Agent Space에서 BADA 리소스 관계를 매핑하고, 과거 배포 실패 이력에서 원인 후보를 조사했습니다.

508개 관계 · RCA 초안

**BADA 리소스 관계: 508개**

→

03

## 의존성 레이스를 IaC로 예방

Secret Version 생성 전 ECS가 기동한 레이스를 식별하고, CloudTrail 검증 후 Terraform depends_on 보강으로 재발을 방지했습니다.

RCA → 검증 → IaC 개선

**장애 이력 분석: 4분 13초**

# Multi-Region EKS GitOps

## 멀티 리전 EKS 클러스터 구축 및 GitOps 기반 배포 자동화

Seoul Production · Osaka Staging · Tokyo DR

**EKS** **GitOps** **Disaster Recovery**

Argo CD · Kustomize · Terraform · Velero · AWS Transit Gateway

# 개요

AWS 서울, 도쿄, 오사카 3개 리전에 EKS 클러스터를 구축하고,
GitOps 기반의 멀티 리전 운영 환경과 DR 구조를 구성한 프로젝트

## Seoul

**Production · Argo CD Hub**

실제 서비스 운영 환경
GitOps 배포 기준을 관리

## Osaka

**Staging · Validation**

운영 전 배포를 검증
환경별 설정과 장애를 사전 확인

## Tokyo

**DR Restore Target**

Velero 기반 백업, 복구 환경
장애 복구 시나리오를 대비

PROJECT 02 | AWS · KUBERNETES · GITOPS

# 설계 및 구현

클러스터 기반을 구성하고, GitOps로 배포하며, 장애 상황의 복구 경로까지 단계적으로 검증했습니다.

01

## 멀티 리전 기반 구성

서울·오사카·도쿄에
역할별 EKS 환경을 구성

**EKS**
**VPC**
**Transit Gateway**

Seoul · Production
Osaka · Staging
Tokyo · DR

→

02

## GitOps 배포 체계

선언형 설정을 기준으로
환경별 배포 흐름을 통일

**Seoul Argo CD Hub**
**Kustomize Overlay**
**원격 클러스터 동기화**

Git → Argo CD → Staging / Production

→

03

## 운영 검증·복구 대비

배포 전 의존성을 검증하고
장애 복구 경로를 준비

**Staging Validation**
**Velero Backup**
**S3 Restore**

Argo CD Self-healing
Velero Backup / Restore

**구축에서 끝나지 않고, 배포와 복구까지 검증하는 멀티 리전 운영 구조**

# GitOps 배포 흐름

하나의 Git 기준 상태를 바탕으로 Staging에서 검증하고, 검토된 변경만 Production에 반영했습니다.

## Git Repository

공통 Base와 환경별 Overlay를
선언형 설정으로 관리

**Base Manifest**
**Kustomize Overlay**
**Git = Source of Truth**

**1.선언 상태 변경 감지 →**

## Seoul · Argo CD Hub

Git 선언 상태를 감지하고
원격 EKS 클러스터와 동기화

**원격 클러스터 동기화**
**Desired State Sync**
**Self-healing**

**2. Staging 동기화 →**

## Osaka Staging

전용 Overlay로 운영 전 배포·의존성을 검증

**Staging Image**
**APP_ENV=staging**
**Validation**

**↓ 3. Staging 결과 확인·리뷰**

## 검증 · 리뷰

Staging 결과를 확인하고
운영 반영 여부를 검토

PR / Commit으로 Git 선언 상태 갱신

**↓ 5. 갱신된 Git 상태 동기화**

## Seoul Production

갱신된 Git 선언 상태를
운영 클러스터에 동기화

**Production Overlay · Desired State**
**Argo CD Sync**

**4. PR / Commit으로 Git 선언 상태 갱신**

**자동화의 목적은 빠른 배포가 아니라, 검증된 Git 상태를 일관되게 운영에 반영하는 것입니다.**

# 장애 유형에 따른 복구 전략

구성 오류는 선언 상태로 복구하고, 데이터 손실은 백업 시점 기준으로 복구합니다.

**장애 발생** → **장애 범위 판단**

**아니오**

## 구성 오류 · Pod 장애

**Pod 삭제 · 설정 드리프트**

**Argo CD Self-healing**

선언 상태와 실제 상태의 차이를 감지해
Pod와 Kubernetes 리소스를 다시 동기화

**자동 재동기화**

**예**

## 데이터 손실

**Namespace 삭제 · PVC 데이터 손실**

**Velero 복구 경로**

자가치유로 복구할 수 없는 데이터·리소스는
사전에 보관한 백업본으로 복구

**백업 기반 복구**

**백업 복구 필요** →

## Tokyo DR 복구

**S3 백업본 기반 Tokyo DR 복구**

**Velero Restore**

백업 시점 기준으로 Kubernetes 리소스와
Namespace/PVC 데이터를 복구·검증

**수동 실행 · 복구 검증**

## Argo CD가 담당하는 것

선언 상태 재동기화 · Pod/설정 복구
데이터 복구는 담당하지 않음

## Velero가 담당하는 것

백업 보관 · Namespace/PVC 복구
복구 시점은 백업본 기준

# Pod 장애 자동 복구 검증 결과

단일 Pod 장애 시 복구 시간과 데이터 영향 범위를 측정했습니다.

**RTO 약 6초**
첫 TIMEOUT → 첫 HTTP 200 응답

**RPO 0**
DB·PVC 미영향 Pod 장애 범위

**수동 개입 0회**
Pod 삭제 후 ReplicaSet·Readiness 기반 자동 복구

## Pod 강제 삭제 기반 Self-healing 검증

| 측정 항목 | 실측 결과 | 의미 |
|---|---|---|
| **장애 주입** | kubectl delete pod | 단일 Pod 장애를 의도적으로 주입 |
| **서비스 영향** | TIMEOUT 6회 / 약 5~6초 | 복구 중 짧은 요청 실패 구간 확인 |
| **복구 시간 (RTO)** | 약 6초 | ReplicaSet 생성 → Scheduler 배치 → 컨테이너 시작 → Readiness 통과 후 트래픽 복구 |
| **데이터 영향 (RPO)** | 0 | 이번 검증은 Pod 장애만 대상으로 하며 DB·PVC 데이터에는 영향을 주지 않음 |
| **최종 상태** | HTTP 200, 약 7ms | 서비스 정상 응답으로 회복 확인 |

**측정 범위**

RPO 0은 데이터 계층에 영향을 주지 않은 단일 Pod 장애 시나리오의 결과입니다.
Namespace·PVC·데이터 손실은 Velero와 S3 백업본을 이용하는 별도 복구 절차로 설계했습니다.

# 기술 스택

## Container / Orchestration

Amazon EKS · Kubernetes · Docker · Helm
Private Node Group · ALB Controller · EBS CSI

## GitOps / Deployment

Argo CD · Kustomize · GitHub Actions
Base Manifest · Overlay · Remote Cluster Sync

## Network / Multi-Region

Amazon VPC · Transit Gateway · ALB
Seoul · Osaka · Tokyo · Private Subnet

## Identity / Configuration

IAM · IRSA (OIDC) · Secrets Manager
Kubernetes ServiceAccount · Parameterized Config

## Backup / Disaster Recovery

Velero · Amazon S3 · Snapshot
Namespace / PVC Restore · Tokyo DR Validation

## Observability / Validation

CloudWatch · Prometheus · Grafana
Staging Validation · Readiness Check · Git-based Review

# 회고 및 배운 점

선언형 배포, 사전 검증, 장애 복구를 하나의 운영 흐름으로 연결했습니다.

01

## 선언형 운영

Git을 기준 상태로 관리하고,
Argo CD가 실제 클러스터 상태를
지속 동기화

**GitOps · Desired State · Self-healing**

02

## 검증 중심 배포

Osaka Staging에서
환경별 설정과 의존성을 검증한 뒤
운영 반영

**Staging · Kustomize · Validation**

03

## 장애 범위별 복구

구성 오류는
Argo CD로 재동기화 하고
데이터 손실은
Velero 백업으로 복구

**Recovery Scope · Velero · DR**

확장한 역량: GitOps 운영 · Staging 기반 배포 검증 · 장애 범위별 복구 설계

# Docker Swarm 기반 Auto Scaling 반응속도 비교 분석

Shell Script와 AWS CloudWatch 방식의 자동 확장 반응속도 비교 분석

Docker Swarm · GitHub Actions · Prometheus · Grafana · CloudWatch

# 개요

EC2 3대 Docker Swarm 환경에서 Shell Script와 AWS CloudWatch 자동확장의 반응속도를 실측 비교했습니다.

## 2 KEY QUESTIONS

### Q1 어느 방식이 더 빠르게 반응하는가?

Shell Script 10초 폴링과 CloudWatch 최소 60초 수집 주기의 반응 시간을 정량 측정

### Q2 그 차이가 실제 운영에서 의미 있는가?

반응 지연을 운영 안정성·관리 비용과 함께 해석해 서비스별 선택 기준으로 정리

## PROJECT COMPONENTS

**인프라 구성**
Manager 1 · Worker 2

**CI/CD**
GitHub Actions · ECR

**모니터링**
Prometheus · Grafana

**Auto Scaling**
Shell Script · CloudWatch

### 실험 조건

| 시작 상태 | 부하 명령 | 반복 횟수 |
|---|---|---|
| Nginx 컨테이너 2개 | stress --cpu 2 /120s | 조건별 10회 정량 |

**실험 환경 | AWS EC2 t3.small × 1, t3.micro × 2, Docker Swarm, Overlay Network**

# 구성 및 운영 흐름

배포·관측·확장 경로를 분리해 동일한 Docker Swarm 환경에서 두 자동확장 방식을 비교했습니다.

## DEPLOYMENT PATH

**GitHub Actions**
빌드 · 배포 자동화

→

**Amazon ECR**
이미지 저장소

↗

**Docker Swarm**
Manager 1 · Worker 2

- **Manager** — t3.small · 제어
- **Worker 1** — t3.micro · 부하
- **Worker 2** — t3.micro · 부하

## AUTO SCALING

### Shell Script 방식

**Prometheus API → 10초 폴링 Shell Script → docker service scale**

### AWS CloudWatch 방식

**CloudWatch Alarm → SNS → docker service scale**

**제어는 Manager에, 부하와 서비스 실행은 Worker에 분리해 자동확장 방식의 반응 경로를 비교했습니다.**

# 자동확장 방식 / 실험 설계

Shell은 서비스 규모 증가, CloudWatch는 Alarm 전환 시점까지의 반응 시간을 동일 조건으로 기록했습니다.

## DETECTION PATHS

01

### Shell Script

**Prometheus API · 10초 폴링**

CPU 50% 초과 → autoscale.sh → 컨테이너 증가 감지

02

### AWS CloudWatch

**EC2 CPU · 최소 1분 수집**

Alarm 상태 전환 감지 → 반응 시간 기록

## EXPERIMENT MATRIX

| 조건 | 방식 | 부하 | 측정 목표 |
|---|---|---|---|
| A | Shell | Worker 1 | 단일 부하 반응 |
| B | Shell | Worker 1·2 | 동시 부하 반응 |
| C | CloudWatch | Worker 1 | 단일 부하 감지 |
| D | CloudWatch | Worker 1·2 | 동시 부하 감지 |

### 공통 측정 조건

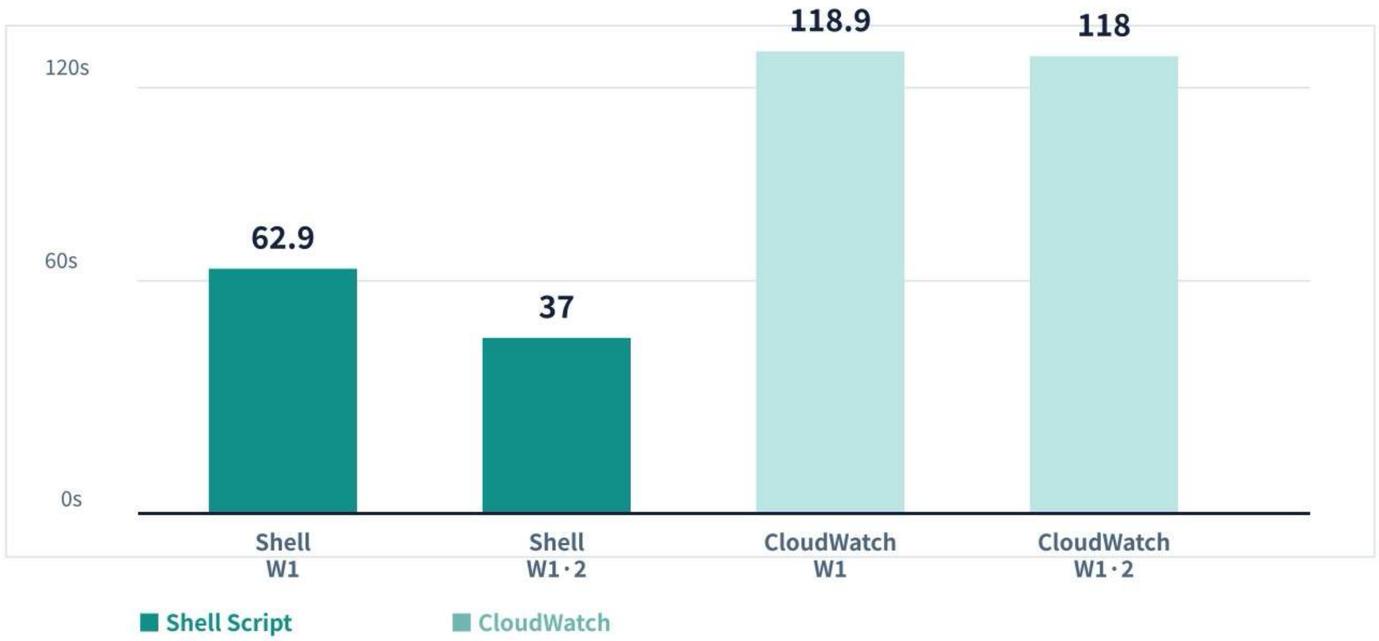
**Nginx 컨테이너 2개 · stress --cpu 2 --timeout 120s · 조건별 10회 반복 · 총 40회**

**동일한 부하 조건을 유지해, 감지 구조가 반응 시간에 미치는 차이를 비교했습니다.**

# 측정 결과

4개 실험 조건을 각각 10회 반복한 평균 반응 시간입니다.

## AVERAGE RESPONSE TIME | 초 (second)




## MEASURED AVERAGES

| 조건 | 부하 | 평균 |
|---|---|---|
| Shell | Worker 1 | **62.9초** |
| Shell | Worker 1·2 | **37.0초** |
| CloudWatch | Worker 1 | **118.9초** |
| CloudWatch | Worker 1·2 | **118.0초** |

### OBSERVATION

**Shell은 동시 부하에서 37.0초로 가장 빠르게 반응**

CloudWatch는 부하 범위와 무관하게 118초 내외로 관측

# 결과 해석

Shell은 빠른 감지, CloudWatch는 관리형 Alarm 경로라는 서로 다른 운영 특성을 확인했습니다.

## TRADE-OFF

### SHELL SCRIPT

**빠른 반응**

**37~63초**

**10초 폴링 기반**

**운영상 강점**
임계값 도달 후 빠르게 서비스 규모를 조정

**운영상 고려**
Manager 실행 프로세스·운영 코드에 의존

### AWS CLOUDWATCH

**관리형 관측 경로**

**118초 내외**

**최소 1분 수집 주기**

**운영상 강점**
관리형 지표·Alarm 기반으로 관측 경로를 분리

**운영상 고려**
수집·Alarm 전환 주기에 따른 구조적 반응 지연

**선택 기준** 저지연 대응이 우선이면 Shell, 관리형 관측·운영 분리가 우선이면 CloudWatch 경로를 검토

# 기술 스택

## Container / Orchestration

Docker Swarm · Docker Engine · Nginx
Manager · Worker · Overlay Network

## CI/CD / Registry

GitHub Actions · Amazon ECR · SSH Deploy
Commit SHA Tag · Swarm Redeploy

## Traffic / Load Balancing

HAProxy · Docker Service · Nginx
Port 80 · Swarm Service Discovery

## Auto Scaling

Bash · autoscale.sh · docker service scale
CPU 50% / 20% · 10초 Polling

## Monitoring / Alert

Prometheus · Node Exporter · Docker Metrics
Grafana · Slack Webhook · Port 9323

## Cloud / Measurement

Amazon CloudWatch · SNS · stress
CSV · 4 Conditions · 40 Runs

# 읽어주셔서 감사합니다.

신뢰할 수 있는 운영 환경을 만드는 구성원으로 성장하겠습니다.

**김종언**

**KIM JONGEON**

CONTACT

vocolate17@gmail.com

010.4270.9729